Group ni Alex

Lunti: Smart Personal Finance Management
Software Project Management Plan
Version 01.00

Revision History

Date	Version	Description	Author
09/12/2026	01.00	Creation of SPMP	Ygot, Enriquez, Birondo, Atayde

Table of Contents

1 Introduction 5

1.1 Purpose of Project Management Plan 5

2 Executive Summary of Project management plan 5

2.1 Assumptions/Constraints 6

3 Scope Management 6

3.1 Work Breakdown Structure 7

3.2 Deployment Plan 8

3.3 Change Control Management 9

4 Schedule/Time Management 10

4.1 Milestones 10

4.2 Project Schedule 11

4.2.1 Dependencies 11

5 Cost/Budget Management 11

6 Quality Management 12

7 Human Resource Management 12

8 Communications Management 13

8.1 Communication Matrix 13

9 Risk Management 14

9.1 Risk Log 14

10 Issue Management 14

10.1 Issue Log 14

11 Procurement Management 14

12 Compliance Related Planning 15

Appendix A: Project Management Plan Approval 16

Appendix B: References 17

Appendix C: Key Terms 18

Software Project Management Plan

1 INTRODUCTION

1.1 PURPOSE OF PROJECT MANAGEMENT PLAN

The purpose of this Software Project Management Plan (SPMP) is to establish the management, organizational, execution, and control framework for the development of **Lunti: Smart Personal Finance Management**. This document defines the project scope, schedule, budget, quality control mechanisms, resource allocations, risk management strategies, and communication protocols required to deliver the Lunti mobile app and backend platform.

The intended audience of the Lunti SPMP includes all project stakeholders: the project team (Atayde, Ygot, Birondo, Enriquez), project evaluators, technical advisors, and administrative overseers.

2 EXECUTIVE SUMMARY OF PROJECT MANAGEMENT PLAN

The Lunti Project Management Plan (PMP) provides a structured approach to delivering a mobile-based personal finance and grocery management application tailored for individuals seeking to manage personal spending and everyday purchases.

The project aims to integrate financial tracking with purchasing and food management to help users make more informed spending decisions, reduce unnecessary purchases, and minimize food waste. Core functionalities include:

- **Expense Management & Budgeting:** Manual expense entry, automatic categorization, and flexible daily, weekly, or monthly budgeting limits.
- **Receipt Scanning and OCR:** Extracts text and purchase information from receipts using Google Cloud Vision for user verification.
- **Smart Replenishment & Shopping List:** Analyzes historical product purchases to predict when regularly purchased products need replenishment and enhances shopping lists.
- **Ingredient & Recipe Recommendations:** Estimates available food ingredients and prioritizes recipes based on current inventory to minimize additional costs.
- **AI-Powered Insights:** Uses the OpenAI API to provide personalized, data-grounded observations about the user's finances.

The PMP covers scope, schedule, quality, risk, communication, resource, and compliance management to ensure successful project execution. Agile methodology with 2-week sprints will guide development, testing, and deployment.

The ultimate goal of Lunti is to eliminate passive financial logging by connecting money to meals, while maintaining security, usability, and high performance standards as outlined in the SRS.

2.1 ASSUMPTIONS/CONSTRAINTS

Assumptions:

- Users will have Android mobile devices with functional cameras and an active internet connection to access AI services, OCR processing, and the cloud backend.
- Supabase and cloud hosting platforms will maintain availability for database hosting, authentication, and backend execution.
- The receipt workflow strictly requires user confirmation or correction before extracted OCR information is saved to ensure data integrity.
- Budget calculations remain strictly deterministic within backend application code, never relying on AI models for core math

Constraints:

- Advanced banking integrations, multi-currency support, family/shared accounts, and nutrition tracking are explicitly outside the MVP scope.
- Lunti operates primarily as a mobile-first application optimized for touch interaction and one-handed use.
- OCR accuracy depends on receipt image quality; poorly lit or folded receipts may require manual correction.
- AI-generated insights are strictly advisory, non-authoritative outputs and must rely on available user data rather than generic advice.

3 SCOPE MANAGEMENT

The Lunti project focuses on developing a React Native mobile application and FastAPI backend that simplifies personal finance and grocery tracking. The project scope covers design, development, testing, and deployment of a secure, responsive, and user-friendly platform capable of:

- **Expense and Budget Management:** Deterministic tracking of transactions and remaining balances, generating alerts for budgets nearing their limits.
- **Receipt Scanning and OCR:** Image capture, storage via Supabase, and text parsing via Google Cloud Vision.
- **Purchase Pattern Tracking:** Computing spending summaries, interval detection, and tracking recurring product frequencies.
- **Smart Replenishment and Shopping Lists:** Suggesting items for repurchase based on interval data and allowing user adjustments.
- **Recipe Matching:** Scoring available ingredients against a recipe catalog and generating cost estimates for missing ingredients.

- **Notifications:** Dispatching push alerts for budgets, replenishments, and meal recommendations via Firebase Cloud Messaging.

Out of Scope:

- Direct integration with bank accounts or credit card APIs.
- Family or shared account functionality.
- Nutrition tracking and multi-currency conversions.

3.1 WORK BREAKDOWN STRUCTURE

1. **Project Initiation**
   1.1 Requirement gathering and analysis (SRS creation)
   1.2 Architecture, data, and interfaces (SDD creation)
   1.3 Project planning and approval (SPMP creation)

2. **UI/UX Design**
   2.1 Wireframes and mockups for Home, Transactions, Analytics, Meals, Budget/Profile
   2.2 Mobile-first responsive design for React Native
   2.3 Accessibility, clear touch targets, and contrast compliance

3. **Backend Development**
   3.1 PostgreSQL database setup on Supabase and Auth integration
   3.2 Implementation of Role-Based Access Control (RBAC)
   3.3 Modular monolith service structuring via Python and FastAPI

4. **AI & OCR Engine Development**
   4.1 Integration of Google Cloud Vision API for receipt parsing
   4.2 Purchase pattern detection and deterministic budget calculations
   4.3 OpenAI prompt engineering for sanitized financial insights

5. **Frontend Development (React Native)**
   5.1 Navigation, floating action buttons, and global state (Zustand)
   5.2 Camera interface and draft transaction forms
   5.3 Budget tracking UI, analytics charts, and shopping list components
   5.4 Notifications and feedback UI integration

6. **Integration and Testing**
   6.1 Functional testing for all domain services
   6.2 Performance testing for image uploads and OCR responses
   6.3 Security testing (Supabase Auth, secure token storage)
   6.4 Usability and mobile responsiveness testing

7. **Deployment**
   7.1 Staging environment setup

7.2 Production deployment (Backend hosting, EAS Build for mobile)
7.3 User acceptance testing

8. **Documentation and Training**
   8.1 User manuals and application guidance
   8.2 Developer documentation (FastAPI/Swagger UI)

9. **Maintenance & Support**
   9.1 Bug fixes and patches
   9.2 Performance monitoring

3.2 DEPLOYMENT PLAN

The deployment of Lunti will follow a structured, phased approach to ensure smooth rollout and minimal disruption to end users.

1. **Environment Assessment**
   - Verify that target devices (iOS/Android) meet OS and connectivity requirements.
   - Ensure Supabase (Database, Storage, Auth) and backend Python hosting platforms are operational and properly configured.
   - Validate that external API services (Google Cloud Vision, OpenAI, Firebase) are accessible and functional.

2. **Pre-Deployment Preparation**
   - Set up a staging environment for integration testing of all modules, including Expense tracking, OCR parsing, Replenishment, and Recipe matching.
   - Configure environment variables for secure API keys and database credentials.
   - Conduct data schema migrations via Alembic to production-ready formats.

3. **Deployment Execution**
   - Deploy the Python/FastAPI backend to the production environment over HTTPS.
   - Connect the React Native frontend to the Supabase backend for authentication and data storage.
   - Perform smoke tests to validate that all major features—especially receipt capture and budget alerts—work as intended.

4. **User Acceptance and Rollout**
   - Conduct limited user acceptance testing (UAT) with sample users representing young professionals and students.
   - Address any feedback or defects regarding OCR accuracy or UI flows before full release.
   - Officially release the mobile application via respective app store channels or testing platforms.

5. **Post-Deployment Support**
   - Monitor application performance and API uptime.
   - Maintain regular backups of user transaction data in PostgreSQL.

3.3 CHANGE CONTROL MANAGEMENT

All changes to the Lunti project will follow a controlled process to ensure that modifications do not negatively affect the system's core deterministic logic, schedule, or quality.

Change Control Process:

1. **Submission of Change Request**
   - Any team member or stakeholder can submit a Change Request if a feature needs to be added, improved, or fixed.
   - The request must include: Description of the change, Reason for the change, Which module is affected (e.g., OCR Orchestration, Expense Service, AI Insights).
2. **Impact Assessment**
   - The Project Manager evaluates how the change affects: Scope, Schedule, and Resources.
3. **Approval**
   - The change is approved or rejected by the project team. Only approved changes are implemented.
4. **Implementation & Testing**
   - Changes are applied in the development environment.
   - Functional testing is strictly done to make sure deterministic financial figures are not corrupted.
5. **Deployment & Documentation**
   - Once tested, the change is deployed. All project documents (SRS, SDD) are updated to reflect the change.
6. **Monitoring**
   - The team checks that the change works as intended in the live system.

Example:

If support for a new receipt format is requested to improve OCR extraction, the team submits a change request, evaluates the effect on schedule, approves it, implements the parsing logic, tests against strict user-confirmation pipelines, and updates the SDD.

4 SCHEDULE/TIME MANAGEMENT

The **Lunti project** schedule will follow a structured timeline to ensure timely completion of all modules, including **Expense Recording, Receipt Scanning and OCR, Budget Management, Spending Analytics, Smart Replenishment, Smart Shopping Lists, Ingredient Management, Recipe Recommendations, and AI-Powered Insights**. The Project Manager (PM) will be responsible for maintaining the schedule, monitoring progress weekly, and updating it whenever there are changes in scope or unforeseen delays.

Schedule Management Approach

- A baseline schedule will be established at the start of the project.
- Progress will be monitored weekly, and variances greater than 10% on key milestones will be addressed immediately.
- All tasks will include dependencies to avoid blocking critical features, such as purchase pattern tracking before smart replenishment, ingredient management before recipe recommendations, and receipt scanning before spending analytics.
- Any changes will be handled using the **Change Control Management** process outlined in Section 3.3.

4.1 MILESTONES

The table below lists the milestones for this project, along with their estimated completion timeframe.

Milestones	Estimated Completion Timeframe
Project Approval & Requirement Analysis	Week 1-2
UI/UX Design Completion (Wireframes & Mobile Layout)	Week 3-4
Backend Setup (FastAPI, Supabase Auth, PostgreSQL)	Week 4-5
OCR Pipeline & Expense Engine Development	Week 5-7
Frontend Development (React Native Views, Forms, Zustand)	Week 6-8
AI Integration (Insights, Replenishment, Recipe Matching)	Week 8-9
Deployment to Staging and User Acceptance Testing	Week 9-10
Final Production Deployment	Week 10-11
Documentation & Project Submission	Week 11-12

4.2 PROJECT SCHEDULE

LUNTI_PROJECT SCHEDULE

4.2.1 Dependencies

- **Design-to-Frontend:** Frontend Development cannot reach its final stages without the completed UI/UX mockups.
- **API Availability:** Project progress depends on the uptime and response speed of third-party Google Cloud Vision and OpenAI APIs.
- **Core-to-Analytics:** The Analytics and Replenishment modules strictly depend on the successful completion of the underlying transactional database logic.

5 COST/BUDGET MANAGEMENT

The development team will implement a structured cost management approach to ensure financial sustainability, efficient resource allocation, and controlled operational expenses throughout the 3-month project lifecycle.

Cost Management Approach

- A baseline budget will be established during project initiation.
- Monthly cost monitoring will track API usage (OpenAI, Google Vision) to prevent overrun.
- A contingency reserve will be allocated for unexpected cloud scaling or operational expenses.

Category	Description	Duration	Cost (P)
Personnel	Project Management / QA Lead	3 Months	105,000
Personnel	Backend / Data Architect	3 Months	90,000
Personnel	Frontend Mobile Developer	3 Months	90,000
Personnel	UI/UX Designer	2 Months	50,000
Infrastructure	Cloud Services (Supabase, API Usage)	3 Months	11,100
Infrastructure	Dev Licenses & Software Tools	Annual / 3 Months	3,000
Operations	Utilities & Miscellaneous	Project Duration	16,000

Operations	Contingency Reserve	Project Duration	32,000
TOTAL PROJECT BUDGET			**P397,100**

6 QUALITY MANAGEMENT

The Lunti team will implement formal Quality Assurance processes. Crucially, the system must validate the separation of deterministic financial calculations from probabilistic AI logic.

Quality Objective	**Target Standard**	**Validation Method**
Functional Correctness	Extracted OCR fields correctly map to UI draft forms.	Functional & System Testing
Defect Prevention	No critical bugs allowing unconfirmed writes.	Defect Tracking & Retesting
UI Consistency	Clear feedback, mobile-optimized touch targets.	UI Inspection & Usability Review
Accurate Calculations	Budget totals strictly calculated deterministically in DB.	Unit Testing & Integration Testing
Performance Efficiency	Responsive client caching via TanStack Query.	Performance Testing
Requirement Compliance	All SRS critical features (Section 3.1) implemented.	Requirements Traceability Matrix (RTM)

7 HUMAN RESOURCE MANAGEMENT

Role	**Key Responsibilities**
Project Manager	Oversees project execution, schedule, budget, and stakeholder communication.
Backend Developer	Develops FastAPI endpoints, Supabase configuration, OCR pipelines, and deterministic math logic.
Frontend Developer	Implements React Native UI, Zustand state, TanStack Query, and camera API integrations.
UI/UX Designer	Designs Figma wireframes, mobile interactions, and user flows.
QA/Test Engineer	Validates requirements matrices, performs integration testing, ensures data confirmation flows.

8 COMMUNICATIONS MANAGEMENT

Communication Type	Purpose	Medium	Frequency
Project Meetings	Discuss progress, tasks, and issues	Video Conference / Meeting	Weekly
Progress Reports	Update stakeholders on project status	Written Report / Email	Bi-weekly
Team Coordination	Day-to-day development communication	Team Messaging Platform	Daily
Milestone Review	Evaluate completion of major deliverables	Presentation / Meeting	Per Milestone
Issue Escalation	Address critical risks or blockers	Direct Communication	As Needed

8.1 COMMUNICATION MATRIX

Stakeholder	Messages	Vehicles	Frequency	Communicators	Feedback Mechanisms
Project Sponsor	Project progress, budget status	Formal Report, Email	Monthly	Project Manager	Review meetings
Project Manager	Task updates, schedule status	Team Meetings	Per Milestone	Development Team	Status discussions
Dev Team	Task assignments, API payload specs	Messaging, Repo	Weekly	Team Leads	Pull request reviews
QA/Test Engineer	Defect reports, OCR validation status	Defect Tracking	Daily	QA Engineer	Defect closure
End Users	Application updates, reminders	Push Notifications	As Triggered	System (Firebase)	In-app feedback

9 RISK MANAGEMENT

Risk Management Approach:

1. **Risk Identification:** Evaluate constraints regarding OCR text clarity and AI data thresholds.
2. **Risk Analysis:** Identify consequences if unconfirmed probabilistic data overwrites deterministic financial tables.
3. **Risk Response Planning:** Enforce strict mandatory confirmation gates on the UI for all AI/OCR outputs.
4. **Risk Monitoring:** Review metrics on OCR success rates and fallback usage during sprints.

9.1 RISK LOG

Risk_Logs - Lunti.xlsx

10 ISSUE MANAGEMENT

Issue Management Approach:

- **Issue Identification:** Any member logging API connection failures or calculation bugs.
- **Issue Documentation:** Recorded in Issue Log with severity, description, and target resolution date.
- **Assessment:** Classified as High, Medium, or Low priority (e.g., Budget calculation errors are High severity).
- **Resolution:** Implemented by backend/frontend leads based on domain boundary.
- **Closure:** Formally closed after verified retesting

10.1 ISSUE LOG

Issue_Logs - Lunti.xlsx

11 PROCUREMENT MANAGEMENT

Procurement Approach: Procurement ensures the acquisition of required API keys, cloud tiers, and development assets.

Category	Description	Procurement Strategy
Hardware	Developer mobile devices (iOS/Android), workstations	Company-issued or approved purchase
Cloud Services	Supabase (Database, Auth, Storage), Hosting	Subscription-based cloud providers (Free Tier / Starter)
Software Tools	Figma, GitHub, React Native toolchain	Licensed or open-source solutions
Mobile Deployment	EAS Build credits, Developer Store accounts	Project budget allocation
External Services	Google Cloud Vision API, OpenAI API	Metered pay-as-you-go service agreements

12 COMPLIANCE RELATED PLANNING

Compliance Area	Requirement	Action Plan
Data Privacy	Privacy of personal financial data	Implement explicit Row-Level Security (RLS) policies in PostgreSQL so users only access their own data.
Cybersecurity	Secure Authentication & Networking	Use Supabase Auth (JWTs) and ensure all mobile-to-backend API traffic operates exclusively over HTTPS.
Accessibility	Mobile Readability Standards	Ensure UI designs follow clear touch targets, sufficient contrast, and understandable iconography.
Financial Accuracy	Deterministic Ledger Integrity	Mandate that all AI/OCR outputs are user-confirmed before committing to the DB to prevent silent financial corruption.

Appendix A: Project Management Plan Approval

The undersigned acknowledge they have reviewed the Lunti **Project Management Plan** and agree with the approach it presents. Changes to this **Project Management Plan** will be coordinated with and approved by the undersigned or their designated representatives.

Signature: _____ Date: _____

Print Name: Alexa F. Atayde

Title: Project Manager-Backend Developer-Frontend Developer

Role: _____

Signature: _____ Date: _____

Print Name: Jacob Isaac Enriquez

Title: AI / Data - Backend Develope

Role: _____

Signature: _____ Date: _____

Print Name: Kristal Maureen V. Birondo

Title: Frontend Developer

Role: _____

Signature: _____ Date: _____

Print Name: Nadine Kate P.Ygot

Title: Frontend Developer

Role: _____

APPENDIX B: REFERENCES

The following table summarizes the documents referenced in this document.

Document Name and Version	Description	Location
Lunti_SRS.pdf (v01.00)	Software Requirements Specification detailing functional capabilities and constraints.	Project Repository
Lunti_SDD.pdf (v01.00)	Software Design Document defining modular backend, React Native UI, and DB schemas.	Project Repository

APPENDIX C: KEY TERMS

The following table provides definitions for terms relevant to this document.

Term	Definition
Term	**Definition**
MVP	Minimum Viable Product; the initial version containing essential Lunti features.
OCR	Optical Character Recognition; used to extract merchant and price text from receipt images.
PWA	Progressive Web Application (Reference to baseline formatting; Lunti uses React Native).
Supabase	Cloud platform used by Lunti for PostgreSQL hosting, authentication, and image storage.
Zustand	Lightweight state management library used for the React Native mobile client.